

研究手法 ⇄ 実装 対応表 (v2025-11-01)

現状のリポジトリ構成に合わせて、代表的な研究手法／ライブラリと、それを再現・近似しているスクリプト／関数／CLI・設定キーを 1 枚に整理しました。**そのまま運用チェックリスト**として使えるよう、検証スクリプトと想定ログポイントも併記しています。

凡例

- **Method:** 研究手法／ライブラリ名（代表論文・実装の系統）
- **What:** 何を推定／生成するか
- **Repo:** 実装ファイルと主関数／エントリポイント
- **Config:** 主な設定キー（`arranger_weights.yaml` など）
- **CLI:** 最小再現コマンド例
- **Verify:** 想定を検証スクリプト／ログ

A. Audio (WAV) 特徴量

Method	What	Repo (関数/入口)	Config	CLI
madmom (RNN+DBN)	ビート / ダウンビート	<code>features_backends.py:</code> <code>extract_beats_madmom()</code> / <code>extract_downbeats_madmom()</code> → <code>stems_features.py:</code> <code>extract_drums_features()</code> / <code>integrate_stem_features()</code>	<code>features_backend:</code> <code>beats: madmom,</code> <code>downbeats: madmom</code>	<code>python ops/stems_features.py --stems <dir> --backend-config configs/arranger_weights.yaml --extend-bars</code>
librosa (強化版)	ハイハット密度 (5–12kHz 限定オンセット)	<code>features_backends.py:</code> <code>extract_hat_density_librosa_enhanced()</code> → <code>stems_features.py:</code> <code>extract_drums_features()</code>	<code>features_backend:</code> <code>hat_density:</code> <code>librosa_enhanced</code>	同上
pyloudnorm (R128)	LUFS (楽曲ラウドネス)	<code>features_backends.py:</code> <code>extract_loudness_pyloudnorm()</code> → <code>stems_features.py:</code> <code>extract_mix_features()</code>	<code>features_backend:</code> <code>loudness:</code> <code>pyloudnorm</code>	同上

Method	What	Repo (関数/入口)	Config	CLI
YAMNet (AudioSet)	HH/ Ride など 打楽器確 率	<code>features_backends.py:</code> <code>extract_hat_density_yamnet()</code> (Phase B で有効)	<code>features_backend:</code> <code>hat_density:</code> <code>yamnet</code>	同上

B. Rhythm / Drums (検索・生成)

Method	What	Repo (関数/入口)	Config	CLI
Pattern Matching (Top-K)	テンポ/ ファミリ/ 密度で類似 グルーヴ検 索	<code>scripts/pattern_matcher.py</code> (Top-K, family, density スコ ア)	<code>arranger_weights.yaml:</code> <code>topk, τ, weight.*</code>	<code>python scripts/ pattern_matcher.py --song- package .../ song_package.yaml</code>
Real Groove 置換	実 MIDI→Plan 変換	<code>scripts/ drums_midi_to_plan_real.py:</code> <code>midi→plan</code> / <code>scripts/ drums_midi_to_plan.py</code> (ゴー ストHH補完)	<code>arranger_weights.yaml:</code> <code>drums.*</code>	<code>python scripts/ drums_midi_to_plan_real.py --midi <src.mid> --bars <bars.parquet> --out drums_plan.json</code>
Safe-Kit / Ghost HH	スカスカ対 策 (ゴー ストHH 22- 28vel、 0.20beat)	<code>scripts/ drums_midi_to_plan.py</code>	<code>arranger_weights.yaml:</code> <code>drums.ghost_hh.*</code>	E2E内で自動

C. Harmony / Chord

Method	What	Repo (関数/入口)	Config	CLI
Chordino / NNLS Chroma (Phase C 以降 で採用想定)	コード推 定・補完	(将来) <code>ops/chordino_bridge.py</code> などで Essentia/Vamp 呼び出しを想 定	<code>features_backend:</code> <code>chroma: essentia vamp- chordino</code>	—
Voicing & Piano/Guitar Plan	ボイ シン グ展 開/ セク シヨ ン別 化	<code>scripts/ generate_piano_strings_plans.py:</code> <code>generate_piano_plan()</code> , <code>generate_strings_plan()</code> / <code>voicing_engine.py</code>	<code>arranger_weights.yaml:</code> <code>heuristics.piano/ strings.*</code>	<code>python scripts/ generate_piano_strings_ --song-dir <dir> --emit emit-strings</code>

D. MIDI 整形・書き出し

Method	What	Repo (関数/入口)	Config	CLI	Verify
PrettyMIDI / music21 実 務運用	Plan→MIDI、 テンポトラッ ク分離、絶対 tick→delta 安全変換	scripts/midi_writer.py (Track0のみ set_tempo / クリップ機 構)	configs/ midi_writer.yaml, plan_humanize.yaml	python scripts/ midi_writer.py --plan full_arrangement.json --config configs/ midi_writer.yaml --out out.mid	get_dow end_tim Strings二 無

E. KPI / 検証・集計

Method	What	Repo (関数/入口)	Config	CLI	Verify
KPI Gate (拡張 版)	密度/ バツ ク ビー ト/ス ウィ ング 等の バー 評価	scripts/kpi_gate_enhanced.py: extract_kpi_from_midi_enhanced()	configs/ gate_prod.yaml (セ クション別KPI含 む)	python scripts/ kpi_gate_enhanced.py --midi full_arrangement.mid --bars bars.parquet -- gate-config configs/ gate_prod.yaml --out kpi.json	Pass率、Fai Top、 epsilon_se 用、bars基
バッチ 検証	曲単 位の SLO 監視	scripts/batch_validate_kpi.sh / aggregate_kpi_reports.py	—	./scripts/ batch_validate_kpi.sh <songs_root> -- parallel 4	suno_kpi_s (json csv m 成

F. Stems / Lyrics / Anchors

Method	What	Repo (関数/入口)	Config	CLI	Verify
Stem 特 徴統合	stems→bar特 徴 (hat_density, drums_active, LUFS)	ops/stems_features.py: main(), integrate_stem_features()	features_backend.*, -- extend-bars	python ops/ stems_features.py -- stems <dir> --bars bars.parquet -- backend-config configs/ arranger_weights.yaml --tempo-bpm <bpm> -- extend-bars	ste bar

Method	What	Repo (関数/入口)	Config	CLI	Verify
Lyric Anchors	ボーカル由来 アンカー →MIDI/配置	<code>anchors_from_vocal.py</code> , <code>enrich_anchors.py</code> , <code>anchors_to_midi.py</code>	<code>arranger_weights.yaml</code> : <code>anchors.*</code>	—	Ly MID

G. パターン学習・推薦 (ML系：任意で段階導入)

Method	What	Repo (関数/入口)	Config	CLI	Verify
品質学 習 (弱 教師)	パターン 品質スコ ア学習	<code>ml/ pattern_quality_learner.py</code> , <code>ml/ pattern_quality_config.py</code>	<code>ml.*</code>	—	A/Bで Top-K 一致 率↑
推薦器 (統 合)	Rhythm/ Harmony パターン 推薦	<code>ml/pattern_recommender.py</code> , 各 <code>*_pattern_recommender.py</code>	<code>arranger_weights.yaml</code> : <code>recommender.*</code>	—	KPI, 一致 率, 主 観評 価

既知のギャップ / 推奨TODO

- **Chordino/Essenstia の直接連携**：Phase C で導入予定。 `ops/chordino_bridge.py` 雛形を追加し、 `arranger_weights.yaml` から選択式にする。
- **jSymbolic 相当のMIDI統計**： `analyze_midi_stats.py` を拡張し、ピッチレンジ、音価ヒストグラム、ポリフォニー度数などの feature pack を出力 (`midi_stats.parquet`)。
- **セクション別KPI適用ロジック**： `kpi_gate_enhanced.py` に `gate_prod.yaml.sections.*` を反映 (現在は設定側のみ)。
- **bars.parquet 標準化**： `start_sec/end_sec/drums_active` を常設 (E2Eで `bars_extended.parquet` をデフォルトに昇格)。

運用チェック (短縮版)

1. **Stems特徴**： `ops/stems_features.py ... --extend-bars` → `bars_extended.parquet` を生成。
2. **Drums Real Groove**： `drums_midi_to_plan_real.py` → KPI 90% 以上。
3. **Plan→MIDI**： `midi_writer.py` (Track0のみ `set_tempo / 600beats clip` 確認)。
4. **KPI**： `kpi_gate_enhanced.py --bars .../bars.parquet` (Downbeatsは使わない)。
5. **集計**： `aggregate_kpi_reports.py` (SLO監視)。

付記

- テンポマップは **Track 0 のみ** `set_tempo`。二重書き込みは PrettyMIDI の時間軸を壊すため厳禁。
- `full_arrangement.mid` の妥当性は `downbeats≈151 / end_time≈482s` をクイック指標に。
- `density too low` は **Ghost HH / Ride計上 / 相対密度**の3点セットで収束。